QSYN_XLR Utility

Objective

QSYN_XLR checks feasibility of your accelerator design alone. Later on, once your accelerator design is complete and ready, it will be synthesized together with the full K5-XBOX platform logic (guidelines will be provided). However, this full synthesis can take over 10 minutes to complete, making it impractical for rapid development and debugging. Thus QSYN_XLR offers a quicker feasibility check of your accelerator design in its development stage.

The utility Ensure the correctness and quality of our logic design code by evaluating:

1. **Synthesizability** Confirming that the design can be translated into logic for implementation.
2. **Feasibility** Ensuring the synthesized design meets predefined constraints on gate count and clock frequency, as specified for each assignment.

Keep in mind that a successful simulation does not ensure synthesizability, as it may mask issues like race conditions and combinatorial loops. Conversely, non-synthesizable code can lead to incorrect and unpredictable behavior during simulation. Therefore, it is advisable to verify synthesizability before running simulations.

Method

To verify synthesizability, we employ an FPGA synthesis flow using the Intel-Altera Quartus tool. Although FPGAs and ASICs have significant differences, the guidelines for writing synthesizable code remain similar. In many cases, the same RTL source code can be used for both FPGA and ASIC implementations.

Initial Setup

In case not already done for HW1: In your BIU-Engineering cloud environment open a terminal anywhere and enter the following command:

```
source /project/tsmc65/shared/qsyn/util/qsyn_install.sh
```

This only needs to be executed once ever, and should take just a few seconds. It will permanently configure your account access to the utility. Once the setup is complete, close the terminal and open a new one to continue with the next steps.

Usage Example

Usage example provided for the *sudx_basic* accelerator , you should change 'sudx_basic' wherever referred to the specific checked accelerator name.

Open a terminal and go to the accelerator folder

```
cd $MY_K5_XLRS/sudx_basic
```

Just in case upon modifications you have additional modules instantiated within your accelerator, edit $MY_K5_XLRS/sudx_basic/sudx_basic.f to list them (this is needed also for simulation)

Invoke sudx_basic accelerator synthesis

To execute the utility on the example design, enter the following commands. Each command may take a few minutes, depending on the design's complexity. Since each command depends on the successful completion of the previous one, they must be run in the given order.

Initially you can ignore 'Warning' related messages as long as the scripts complete execution properly and generates valid results.

Check synthesizability report critical errors, and logic resources consumption.

```
qsyn_xlr sudx_basic -syn
```

For our FPGA configuration the limit per accelerator is in the range 20K reported logical-elements.

Check feasibility performers 'place and route' fitting into the device.

```
qsyn_xlr sudx_basic -fit
```

in case you are getting a message:
"ERROR qsyn_output_files/sudx_basic.fit.rpt not generated"
This relates to an environment problem we are currently debugging, to work around run with -all as described below which runs all steps and does not seem to be sensitive to this environment problem.

This step takes most of the compile time and can last several few or more minutes depending on complexity, if it doesn't complete within about 15 minutes per accelerator, most likely your design feasibility is questionable.

Check Timing Perform Static Timing Analysis (STA).

```
qsyn_xlr sudx_basic -sta
```

By default we synthesize to 50MHz, initially attempt to meet this or better speed, and in case noy achieved fix your design timing bottle neck, the method for reporting and analyzing the critical timing path will be provided separately.

Alternatively you can run all three above checks by:

```
qsyn_xlr sudx_basic -all
```

Optional advanced Quartus GUI usage

The Quartus utility also offers a GUI interface for additional design analysis and visualization. Using the GUI is not required for your assignments, but if you'd like to explore your design further, you can open the GUI and load your project as described below.

The QSYN flow mentioned above also generates a file named `<ProjectName>.qpf` (e.g., `max4.qpf` in the example above). To open the Quartus GUI, use the following command in a terminal:

```
$QBIN/quartus
```

Pull down File->Open ... and browse to and your project ".qpf file:

A basic tutorial of the GUI usage is available here:

Intel Quartus Software – GUI Introduction - Part 1 of 6